

[RVHS/2025/Q1]

1 Fish

Name your Jupyter Notebook as

Task_1_<class>_<index number>_<name>.ipynb

For each of the Task 1.1 to 1.3, complete the codes in Jupyter Notebook and add a comment statement at the beginning of the code using the hash symbol #, to indicate the sub-task the program code belongs to, for example

In [1]:

#Task 1.1

Program code

Output:

Task 1.1

Implement the class `Fish` based on the information described below.

[2]

Class Fish	
Attributes	
name	<string> It stores the name of the fish. <i>You can assume that the names used to create Fish instances are always unique.</i>
size	<integer> It stores the size of the fish.
Functions	
Getters	The getter functions for all the attributes of the class Fish.
__str__()	This function returns a string that shows all its attributes. You should allocate fixed 20 and 4 spaces for the name of the fish and size respectively. For example: <pre>"Name:Giant Trevally Size: 33"</pre>
consume(fish)	This function takes a Fish instance <code>fish</code> as input and compares its size with <code>self.size</code>. If <code>fish</code> has a larger size than <code>self.size</code>, it returns a new Fish instance with its name equal that of <code>fish</code>. Otherwise, it returns a new Fish instance with its name equal that of <code>self</code>. The size of this new Fish instance created in both cases are equal to the sum of the size of <code>fish</code> and <code>self</code>. For example: <pre>>>> f1 = Fish("Giant Trevally", 33) >>> f2 = Fish("Pollock", 59) >>> f3 = f1.consume(f2) >>> str(f3) "Name:Pollock Size: 92"</pre>

[RVHS/2025/Q1 Cont']

Task 1.2

Implement the class `Node` and class `SortedLinkedListForFish` using the **free space list** based on the information described below.

Class <code>Node</code>	
Attributes	
<code>data</code>	It stores the data in the node. For this task, <code>data</code> is either <code>None</code> or a <code>Fish</code> instance.
<code>next_ptr</code>	<integer> It stores the array index that leads to the next node.
Functions	
<code>__init__(data)</code>	This constructor initiates the <code>Node</code> attributes as follow: • Assign the argument <code>data</code> to attribute <code>data</code> where <code>data</code> is either <code>None</code> or a <code>Fish</code> instance. • Assign <code>-1</code> to <code>next_ptr</code>

Class <code>SortedLinkedListForFish</code>	
Attributes	
<code>start</code>	<integer> It stores an index to the <code>nodes</code> array which represents the first node of the sorted linked list. <code>-1</code> means the sorted linked list is empty.
<code>next_free</code>	<integer> It stores an index to the <code>nodes</code> array which represents the position of the first-free-to-use node instance. <code>-1</code> means there is no free node available.
<code>count</code>	<integer> It stores the total number of <code>Fish</code> instances stored by the linked list.
<code>nodes</code>	It stores all used and unused <code>Node</code> instances which are linked up by the <code>start</code> and <code>next_free</code> pointers respectively.
Functions	
<code>__init__()</code>	This constructor initiates the sorted linked list attributes as follow: • Assign <code>-1</code> to <code>start</code> • Assign <code>0</code> to <code>next_free</code> • Assign <code>0</code> to <code>count</code> • Append <code>50</code> <code>Node</code> instances to <code>nodes</code>. You should use <code>Node(None)</code> to create these <code>Node</code> instances. • Link up these <code>50</code> <code>Node</code> instances under the <code>next_free</code> pointer.
<code>insert(fish)</code>	This procedure inserts <code>fish</code> which is a <code>Fish</code> instance into the logical sorted linked list (as its node's data) based on the size of <code>fish</code> in ascending order. Upon successful insertion, increase attribute <code>count</code> by <code>1</code> .

[RVHS/2025/Q1 Cont']

<code>delete(fish_name)</code>	This procedure deletes the node from the logical linked list where its attribute <code>data</code> contains a <code>Fish</code> instance that has attribute <code>name</code> equal to <code>fish_name</code> . Upon successful deletion, decrease attribute <code>count</code> by 1.						
<code>display()</code>	This function displays all the data in sorted linked list headed by <code>start</code>. You should make use of <code>str(data)</code> to display the information of the fish instance. For example, <pre>>>> fll = SortedLinkedListForFish() >>> fll.insert(Fish("Tuna", 9)) >>> fll.insert(Fish("Tiger Shark", 40)) >>> fll.insert(Fish("Giant Trevally", 33)) >>> fll.insert(Fish("Skipjack Tuna", 73)) >>> fll.delete("Giant Trevally") >>> fll.display()</pre> <table> <tr> <td>Name:Tuna</td><td>Size: 9</td></tr> <tr> <td>Name:Tiger Shark</td><td>Size: 40</td></tr> <tr> <td>Name:Skipjack Tuna</td><td>Size: 73</td></tr> </table>	Name:Tuna	Size: 9	Name:Tiger Shark	Size: 40	Name:Skipjack Tuna	Size: 73
Name:Tuna	Size: 9						
Name:Tiger Shark	Size: 40						
Name:Skipjack Tuna	Size: 73						

Task 1.1

Implement the class `Fish`.

[2]

Task 1.2

Implement the class `SortedLinkedListForFish`. Use the test case provided for task 1.2 in the file *"useful_code.txt"* to test your implementation.

[17]

Task 1.3

Class <code>SortedLinkedListForFish</code>	
Additional Functions	
<code>pick2RandomFish()</code>	This function returns a tuple that contains two random fish instances that can be found in the sorted linked list. You must implement in such a way that all fish instances in the linked list have equal chance of being picked.
<code>preyTime()</code>	This function first uses <code>pick2RandomFish()</code> and let one of the 2 fish instances picked to consume the other. The two random fish instances picked are removed from the linked list and the new fish instance is inserted to the linked list.

Your task is to:

- Implement the two additional functions in the Class `SortedLinkedListForFish`.
- Write a function `task1_3()` to:
 - read all the fishes in *"fishes.txt"* and insert them into a sorted linked list.
 - repeatedly call `preyTime()` until there is only 1 fish instance left in the sorted linked list.
 - display the information of this last fish using `str(fish)`
- Call `task1_3()` 10 times.

[11]